

Git Workflow Guide

This document describes the Git branching strategy and pull request workflow for the Kinetic Coaching Backend project.

Branch Structure

The project follows a **Git Flow** inspired workflow with two main branches:

Main Branches

Branch	Purpose	Stability
production	Production-ready code	 Protected, stable
development	Integration branch for development	 Testing, pre-production

Feature Branches

All development work happens in **feature branches** created from **dev**.

Workflow Steps

Start Working on an Issue

Before starting any work:

1. **Ensure you're on the latest dev branch:**

```
git checkout dev
git pull origin dev
```

2. **Create a new branch** following the naming convention:

```
git checkout -b TaskType/FeatureName/GH#IssueNumber
```

Example:

```
git checkout -b feat/Add-auth-endpoints/GH#12
git checkout -b fix/Fix-email-validation/GH#45
git checkout -b doc/Add-server-running-README-file/GH#28
```

 For detailed branch naming conventions, see [GitHub Branch & Commits Guide](#)

2 Work on Your Feature

1. **Make your changes** following project coding standards
2. **Commit frequently** using proper commit conventions:

```
git add .  
git commit -m "feat(auth) ✨ add login endpoint"
```

3. **Keep your branch updated** with **dev**:

```
git fetch origin dev  
git rebase origin/dev
```

💡 Regular rebasing prevents merge conflicts

3 Open a Pull Request

When your work is complete:

1. **Push your branch** to the remote repository:

```
git push origin TaskType/FeatureName/GH#IssueNumber
```

2. **Create a Pull Request** on GitHub:

- **Base branch:** **dev**
- **Compare branch:** Your feature branch
- **Title:** Clear description of the changes
- **Description:**
 - Link the related issue (e.g., **Closes #12**)
 - Describe what was implemented
 - Mention any breaking changes or dependencies

3. **Request a review** from at least one team member

4 Code Review Process

✓ Requirements for Merging

- ✓ **Minimum 1 approval** required from a team member
- ✓ All CI/CD checks passing (if configured)
- ✓ No merge conflicts with **dev**
- ✓ Reviewer's feedback addressed

📋 Review Checklist

Reviewers should check:

- ☐ Code follows project conventions
- ☐ Commit messages follow the convention
- ☐ No unnecessary files or code
- ☐ Proper error handling
- ☐ Tests included (if applicable)
- ☐ Documentation updated (if applicable)

5 Merge to Dev

Once approved:

1. **Squash and merge** or **Merge** (team preference)
2. **Delete the feature branch** after merging
3. **Close the related GitHub Issue**

Release to Production

When Dev is Stable

Once all features are tested and **dev** is stable:

1. **Create a Pull Request** from **dev** to **prod**
2. **Thorough review** by team leads
3. **Merge to prod** when approved
4. **Tag the release** (optional):

```
git checkout prod
git pull origin prod
git tag -a v1.0.0 -m "Release version 1.0.0"
git push origin v1.0.0
```

Branch Protection Rules

Production Branch (**prod**)

- ☒ Requires pull request reviews before merging
- ☒ Requires status checks to pass
- ☒ No direct pushes allowed
- ☒ No force pushes allowed

Development Branch (**dev**)

- ☒ Requires at least **1 approval**
- ☒ No direct pushes to **dev** (work in feature branches)

Visual Workflow

```
prod (production-ready code)
  ↑
  | PR with thorough review
  |
dev (integration branch)
  ↑
  | PR with 1+ approval
  |
feature branches (TaskType/FeatureName/GH#IssueNumber)
  ↑
  | Regular commits
  |
local work
```

🔗 Best Practices

☑ DO

- ✓ **Always** create feature branches from the latest **dev**
- ✓ **Keep branches focused** on a single issue
- ✓ **Rebase regularly** to stay updated with **dev**
- ✓ **Write descriptive PR descriptions**
- ✓ **Link PRs to GitHub Issues**
- ✓ **Respond to review feedback promptly**
- ✓ **Delete branches** after merging

✗ DON'T

- ✗ **Don't** push directly to **dev** or **prod**
- ✗ **Don't** work on multiple unrelated features in one branch
- ✗ **Don't** create PRs with unresolved merge conflicts
- ✗ **Don't** merge your own PRs without approval
- ✗ **Don't** leave branches unmerged for too long

🔧 Useful Git Commands

Update Your Branch with Dev

```
# Fetch latest changes
git fetch origin dev

# Rebase your branch
git checkout your-branch
git rebase origin/dev

# If conflicts, resolve them and continue
git rebase --continue
```

```
# Force push (after rebase)
git push --force-with-lease origin your-branch
```

Sync Local Branches

```
# Update dev locally
git checkout dev
git pull origin dev

# Update prod locally
git checkout prod
git pull origin prod
```

Clean Up Old Branches

```
# Delete local branch
git branch -d branch-name

# Delete remote branch (if not auto-deleted)
git push origin --delete branch-name

# Prune deleted remote branches locally
git fetch --prune
```

Related Documentation

- [GitHub Branch & Commits Guide](#) - Detailed commit and branch naming conventions
- [Contributing Guide](#) - General contribution guidelines

Questions?

If you have questions about the workflow, ask in the team channel or check with your team lead.

Remember: Good Git practices lead to a clean history, easier collaboration, and better project maintenance!

